



Swarm • P3

component design • open / untrusted tier

phase 3 of 6

security = pure decisions

COMPONENT DESIGN • PHASE 3

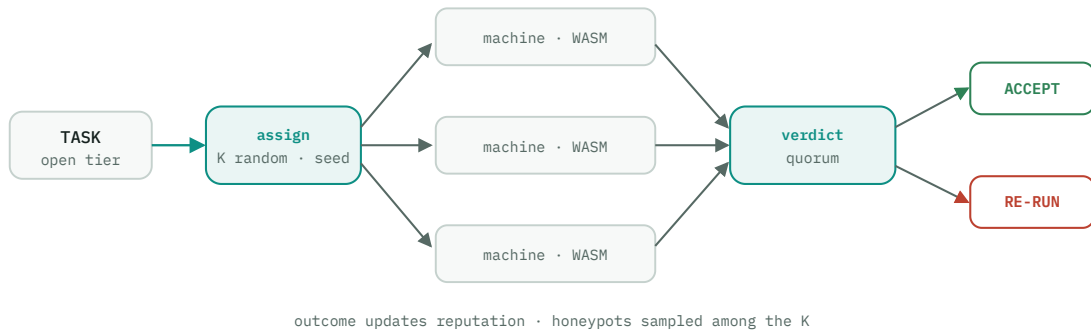
Untrusted machines, trustworthy results.

P3 opens the swarm to anonymous, untrusted machines by running work in a **WASM sandbox** and confirming every result by **quorum**.

The security logic — random assignment, the quorum verdict, reputation, honeypots — is *all pure decision*, so the safety of untrusted compute reduces to a handful of property tests.

01 The open-tier verification path

The core tier (P0–P2) runs a task *once* on a trusted machine. The open tier can't: an untrusted machine may lie. So an open-tier task takes a different path — it's dispatched to **K randomly-chosen** machines, each runs it **sandboxed in WASM**, and the results feed a **quorum verdict** that accepts only the agreed answer. Every box on the decision path is a pure function; only dispatch and execution are I/O.



Run it K ways, believe the majority. An untrusted machine can lie about its own task, but it can't control who else was randomly assigned it — so a minority of liars is outvoted. Assignment, the verdict, and the reputation update are pure; only the dispatch and the sandboxed runs touch the world.

02 Components

WASM sandbox runner

AGENT · DELTA FROM P0

Responsibility — on open-tier machines, run the task inside a Wasmtime sandbox with least-privilege WASI, after checking its signature. P0's run state machine is unchanged; only *how* a task executes swaps from native process to WASM.

◆ FUNCTIONAL CORE · PURE

```
func grants(task Task) WasiCaps
  // least privilege: only what the task declared
func verifyModule(mod []byte, sig Sig, key PubKey) bool
```

▮ IMPERATIVE SHELL · I/O

- instantiate Wasmtime with the computed `grants`
- run the module; capture output
- refuse to run if `verifyModule` is false

Boundary — task + module + signature in; capabilities / a boolean out. The sandbox *policy* is a pure decision; only Wasmtime instantiation is I/O, so "a task gets exactly the caps it declared, no more" is a unit test.

Result verification

VERIFICATION · NEW

Responsibility — send each open-tier task to K randomly chosen machines and accept only the answer a quorum agrees on. This is the heart of untrusted correctness.

◆ FUNCTIONAL CORE · PURE

```
func assign(t TaskID, pool []MachineID,
            k int, seed uint64) []MachineID
func redundancy(tier Tier, rep Reputation) int
func verdict(rs []Result) Verdict
// Verdict = Agreed{value} | Disputed | Insufficient
```

└ IMPERATIVE SHELL · I/O

- dispatch the task to the K machines; collect results with a timeout
- on `Agreed`, accept; else re-run with a fresh assignment

Boundary — task + pool + seed in; a K-subset out; results in; a `Verdict` out. "K=5, 3 honest agree, 2 lie → `Agreed` on the honest value" is a pure test — untrusted correctness, proven without an adversary.

Reputation gating

VERIFICATION · NEW

Responsibility — track how much each identity has earned trust, and set how much verification a machine needs. A Sybil earns no shortcut — every fresh identity starts at zero.

◆ FUNCTIONAL CORE · PURE

```
func update(rep Reputation, agreed bool) Reputation
    // rises on honest agreement, falls on a lie
func weight(rep Reputation) float
func needsK(rep Reputation, tier Tier) int
    // more replicas for low-reputation identities
```

▮ IMPERATIVE SHELL · I/O

- persist the reputation store, keyed by SPIFFE identity
- read on assignment, write after each verdict

Boundary — reputation + agreement in; new reputation / weight / K out. Monotonicity ("honest work never lowers reputation") is a property test; the zero-start rule is what makes faking many identities pointless.

Honeypot spot-checks

VERIFICATION · NEW

Responsibility — occasionally hand out a task whose answer is already known, and blacklist any identity that lies about it. No account needed to catch a liar.

◆ FUNCTIONAL CORE · PURE

```
func shouldProbe(rep Reputation, rng float) bool
func check(claimed Result, known Result) Probe
    // Probe = Match | Lie
func onLie(id SpiffeID) Action    // Blacklist
```

▮ IMPERATIVE SHELL · I/O

- inject a known-answer task at the chosen rate
- compare the claim; blacklist the identity/origin on a lie

Boundary — reputation + rng in, a probe decision out; claimed vs known in, an action out. All pure — "a lie on a known answer is always caught" is a one-line test.

Anonymous enrollment & signing

SECURITY · DELTA

Responsibility — let anyone join the open tier without an account (optionally gated by proof-of-work) and refuse unsigned workloads. Extends the P0/brief enrollment for the untrusted case.

◆ FUNCTIONAL CORE · PURE

```
func admitOpen(req JoinReq, pow PowProof,
               cfg PowCfg) Admit
    // verify PoW; Accept{open-tier id} | Reject
func verifySignature(wl []byte, sig Sig, key PubKey) bool
```

▮ IMPERATIVE SHELL · I/O

- SPIRE issues the open-tier mTLS identity on **Accept**
- distribute signing public keys; verify before dispatch

Boundary — join request + PoW in; an admit decision out. Verifying a proof-of-work or a signature is pure crypto given the inputs; only issuing the certificate is I/O.

03 The safety guarantees are property tests

Because every security decision is a pure function, the properties that make untrusted compute safe are checkable in a test harness — no adversarial cluster, no live attacker. These are the assertions that P3 stands or falls on:

verdict A minority of liars can't flip an **Agreed** result when the honest machines hold the majority of the K-sample.

assign Assignment is uniform over the pool for any seed — so colluding Sybils rarely co-occupy a single task's K-set.

update

Reputation is monotonic in honest agreement, and every fresh identity starts at zero — faking many identities buys nothing.

honeypot

A lie on a known answer is always caught and always blacklists the lying identity.

grants

A task receives exactly the WASI capabilities it declared and no others.

This is why the tier is safe to open. The brief argued the untrusted-machine threat collapses to "lie about your own task, possibly at scale." P3 answers each of those with a pure decision whose guarantee is a unit test — the safety isn't hoped-for, it's asserted.

04 What P3 defers

P3 opens the swarm to the public without moving the FCIS line: a new execution path (WASM), plus verification cores (assign, verdict, reputation, honeypot) and their thin shells. The last phase, **P4**, is **scale infrastructure** — the FoundationDB registry, observability aggregation, rolling upgrades, and the sustained 1M-node validation — followed by production hardening (P5) and self-tuning (P6). None of them change a driver `step` or a verdict; they harden the shell and scale the stores underneath.